
Introduzione a Maven

Il problema che Maven risolve

Quando si sviluppa un progetto Java semplice, spesso si compila così:

```
javac Main.java  
java Main
```

Questo funziona per programmi molto piccoli.

Ma nei progetti reali emergono rapidamente alcuni problemi:

- il progetto contiene **molti file .java**
- servono **librerie esterne**
- bisogna **compilare sempre nello stesso ordine**
- bisogna creare **file JAR**
- bisogna **gestire versioni delle dipendenze**

Fare tutto manualmente diventa difficile.

Per questo esistono i **build tool**.

Un build tool è un programma che:

- compila il codice
- scarica librerie
- esegue test
- crea il file finale (JAR o WAR)

Maven è uno dei build tool più usati nel mondo Java.

Cos'è Maven

Maven è uno strumento che:

1. **gestisce la compilazione**
2. **scarica automaticamente le librerie**
3. **standardizza la struttura del progetto**
4. **crea il file finale (JAR/WAR)**

Maven lavora leggendo un file di configurazione chiamato:

```
pom.xml
```

POM significa:

Project Object Model

È il file che descrive il progetto.

Struttura standard di un progetto Maven

Maven usa una **struttura standard delle directory**.

Questo è molto importante perché Maven si basa su convenzioni.

Struttura tipica:

```
progetto/  
  pom.xml  
  src/  
    main/  
      java/  
      resources/  
    test/  
      java/
```

Significato:

src/main/java codice dell'applicazione

src/main/resources file di configurazione

src/test/java test automatici

Installare Maven

Scaricare Maven dal sito ufficiale:

<https://maven.apache.org/download.cgi>

Scompattare e aggiungere la cartella **bin** al PATH.

Verificare l'installazione:

```
mvn -version
```

Se funziona, Maven è pronto.

Creare un progetto Maven

Creare una cartella e inizializzare un progetto:

```
mvn archetype:generate
```

Oppure creare manualmente la struttura e il file `pom.xml`.

Il file pom.xml

Il file più importante di Maven è:

```
pom.xml
```

Contiene informazioni sul progetto.

Esempio minimale:

```
<project>
  ...
</project>
```

Dentro questo file si definiscono:

- nome del progetto
 - versione
 - librerie
 - configurazione compilazione
-

Concetti fondamentali di Maven

GroupId

Identifica l'organizzazione.

Esempio:

```
com.example
```

ArtifactId

Nome del progetto.

Esempio:

```
myproject
```

Version

Versione del progetto.

Esempio:

```
1.0
```

Dipendenze

Maven può scaricare automaticamente librerie da Internet.

Ad esempio:

- JUnit
- Jackson
- Apache Commons

Le librerie vengono scaricate da:

Maven Central Repository

<https://repo.maven.apache.org/maven2/>

Esempio di dipendenza

Nel pom.xml si scrive:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
  </dependency>
</dependencies>
```

Quando si compila il progetto, Maven scarica automaticamente la libreria.

Comandi Maven più importanti

Tutti i comandi si eseguono nella cartella del progetto.

Compilare il progetto

```
mvn compile
```

Compila i file Java.

Eseguire i test

```
mvn test
```

Esegue i test automatici.

Creare il JAR

```
mvn package
```

Produce il file JAR dentro:

```
target/
```

Pulire il progetto

```
mvn clean
```

Cancella i file compilati.

Build completa

Il comando più usato è:

```
mvn clean package
```

Significa:

1. pulire
 2. compilare
 3. testare
 4. creare il JAR
-

Dove Maven salva le librerie

Maven salva le dipendenze nella cartella:

```
~/.m2/repository
```

Su Windows:

```
C:\Users\USERNAME\.m2\repository
```

Questo evita di scaricare le librerie ogni volta.

Maven in IntelliJ IDEA

IntelliJ ha supporto Maven integrato.

Creare progetto Maven:

File → New Project → Maven

IntelliJ:

- legge il pom.xml
- scarica automaticamente le dipendenze
- mostra i comandi Maven nella finestra "Maven"

Comandi disponibili direttamente nell'interfaccia:

- compile
- test
- package
- install

Quando si modifica il pom.xml, IntelliJ aggiorna automaticamente il progetto.

Maven in Visual Studio Code

In VS Code serve installare alcune estensioni.

Aprire Extensions e installare:

Extension Pack for Java Maven for Java

Queste estensioni permettono:

- riconoscere progetti Maven
- eseguire goal Maven
- gestire dipendenze

Nel pannello Maven di VS Code si possono eseguire comandi come:

```
compile  
test  
package
```

oppure usare il terminale.

Esempio di pom.xml didattico

Questo pom.xml funziona nella maggior parte dei progetti Java didattici.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
    http://maven.apache.org/xsd/maven-4.0.0.xsd">  
  
  <modelVersion>4.0.0</modelVersion>
```

```
<groupId>org.example</groupId>
<artifactId>java-project</artifactId>
<version>1.0-SNAPSHOT</version>

<properties>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>

  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>

</dependencies>

<build>
  <plugins>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
    </plugin>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.1.2</version>
    </plugin>

  </plugins>
</build>

</project>
```

Come usare questo pom.xml

1. creare cartella progetto
2. inserire pom.xml
3. creare cartella codice

src/main/java

4. inserire file Java

Esempio:

```
src/main/java/org/example/Main.java
```

Esempio di compilazione

Da terminale nella cartella del progetto:

```
mvn clean compile
```

Creare il JAR

```
mvn package
```

Il file sarà in:

```
target/java-project-1.0-SNAPSHOT.jar
```

Eseguire il programma

Se il JAR contiene la classe principale configurata, si può eseguire:

```
java -jar target/java-project-1.0-SNAPSHOT.jar
```

Comandi Maven più usati (riassunto)

Compilare:

```
mvn compile
```

Test:

```
mvn test
```

Creare JAR:

```
mvn package
```

Pulire progetto:

```
mvn clean
```

Build completa:

```
mvn clean package
```

Idea importante da ricordare

Maven introduce tre concetti fondamentali:

1. **convention over configuration** i progetti seguono una struttura standard
2. **gestione automatica delle dipendenze**
3. **build riproducibile**

Questo significa che qualsiasi sviluppatore può aprire il progetto e costruirlo con un solo comando:

```
mvn package
```

Alcuni riferimenti

Apache Maven – sito ufficiale <https://maven.apache.org/>

Guida Maven <https://maven.apache.org/guides/getting-started/index.html>

Maven Central Repository <https://repo.maven.apache.org/maven2/>

Documentazione Maven in IntelliJ <https://www.jetbrains.com/help/idea/maven.html>

Documentazione Maven per VS Code https://code.visualstudio.com/docs/java/java-build#_maven